

LLM-based UAV Path Planning for Autonomous and Adaptive Industry Systems

WENJING XIAO*, CHENGLONG SHI*, and MIAOJIANG CHEN[†], School of Computer, Electronics and Information, Guangxi University, and the Guangxi Key Laboratory of Multimedia Communications and Network Technology, China

ATHANASIOS V. VASILAKOS, Dept of ICT and Center for AI Research, University of Agder, Norway

MIN CHEN, School of Computer Science and Engineering, South China University of Technology, China

AHMED FAROUK, Faculty of Computers and Artificial Intelligence, Hurghada University, Egypt

In the era of Industry 5.0, Unmanned Aerial Vehicles (UAVs) equipped with multiple sensors play a vital role in industrial tasks such as patrol and surveillance, offering distinct advantages of high mobility, accurate perception, and autonomous operation. However, traditional path planning methods for UAVs struggle with challenges related to interpretability and adaptation to dynamic industrial environments. To address these challenges, this paper proposes a novel LLM-based approach to UAV swarm path planning in autonomous and adaptive industrial systems. The proposed method aims to minimize the time and computational consumption of path planning while improving the task completion rate of UAVs. Specifically, we first propose the multi-step deep thinking movement decision samples generation algorithm (MSDTMD-SG) to generate deep thinking training samples for LLMs at different ends and fine-tune them. Second, we design a scene memory and replay learning mechanism, enabling damaged UAVs to store perceived information and generate training samples via MSDTMD-SG for continuous LLM learning and optimization. Finally, extensive experiments demonstrate that the proposed method exhibits strong adaptability to dynamic environments, achieves the highest task completion rate among all methods, and maintains competitive system consumption.

CCS Concepts: • **Computing methodologies** → **Planning under uncertainty**.

Additional Key Words and Phrases: Large language model, UAV swarm path planning, cloud-edge collaboration, industry system

*Both authors contributed equally to this research.

[†]Corresponding author.

This work was supported in part by the National Natural Science Foundation of China under Grant 6250071921; in part by the Natural Science Foundation of Guangxi, China, under Grant 2025GXNSFBA069394 and Grant 2025GXNSFAA069958.

Authors' Contact Information: Wenjing Xiao, wenjingx@gxu.cn; Chenglong Shi, chenglongs@st.gxu.edu.cn; Miaojiang Chen, mjchen_cs@gxu.edu.cn, School of Computer, Electronics and Information, Guangxi University, and the Guangxi Key Laboratory of Multimedia Communications and Network Technology, Nanning, China; Athanasios V. Vasilakos, thanos.vasilakos@uia.no, Dept of ICT and Center for AI Research, University of Agder, Grimstad, Norway; Min Chen, minchen@ieee.org, School of Computer Science and Engineering, South China University of Technology, Guangzhou, China; Ahmed Farouk, ahmed.farouk@sci.svu.edu.eg, Faculty of Computers and Artificial Intelligence, Hurghada University, Hurghada, Egypt.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1556-4703/2025/11-ART

<https://doi.org/10.1145/3774655>

1 INTRODUCTION

Industry 5.0 is widely regarded as the next stage in industrial evolution, emphasizing human-centricity, sustainability, and resilience [35]. As UAV-related technologies continue to advance, UAVs are playing an increasingly vital role in diverse industrial applications, including unmanned delivery, farmland operations, low-altitude photography, inspection, and disaster rescue [20], [31], [23]. In industrial scenarios, optimizing UAV mobility paths and quality of service (QoS) has been a significant area of research. To enhance task execution efficiency, multiple UAVs are often deployed in swarms to collaboratively complete missions. Ensuring safe movement and efficient task execution for each UAV, especially in dynamic environments is critical. This necessitates careful planning of movement paths to guarantee that each UAV reaches its assigned location or navigates safely without collisions, either with other UAVs or low-altitude obstacles. Traditional UAV swarm path planning methods, based on Deep Neural Networks (DNNs) or heuristic algorithms, face challenges related to limited interpretability and poor adaptability in dynamic environments [38]. While DNN-based approaches leverage constraints and loss functions for extensive training [36], these methods typically allow us to infer the reasons for poor performance only through experience or ablation studies, rather than enabling the model to learn and correct its errors autonomously, as humans do during the learning process. Similarly, heuristic algorithms often struggle to adapt flexibly to complex, dynamic scenarios and achieve optimal solutions.

As General Artificial Intelligence (GAI) becomes a hot research topic, LLMs have gained widespread attention and become increasingly popular in text generation applications. This is because LLMs generate interpretable text in a human-understandable manner, thanks to training on vast amounts of foundational knowledge [21]. The generative capability of LLMs enables them to produce human-understandable content to guide UAV swarm movement in robust EC scenarios [13]. However, this also comes with challenges and limitations, as LLMs require substantial computational and storage resources during training and inference [40], which presents a significant challenge in resource-constrained edge environments. Fortunately, compression and quantization techniques for LLMs significantly improve inference speed and reduce resource consumption without significantly impacting model performance, enabling efficient and stable deployment in edge scenarios.

We study a low-altitude UAV swarm rescue scenario, as illustrated in Fig. 1, where each UAV is equipped with a lightweight quantized LLM, while a large LLM is deployed on the cloud server. The UAVs in the UAV swarm act as mobile edge devices that need to move from their starting positions to the same destination to provide rescue services in the shortest time without collisions. During movement, each UAV can perceive and utilize the local environmental information around it to move step by step until it reaches the destination. When the UAV has good communication conditions with the cloud, it will turn on its maximum range perception and upload the perceived information to the cloud for motion inference to ensure that it moves faster to its destination, and when the communication conditions are poor or it is not able to communicate with the cloud, the UAV will take the initiative to reduce the perceived range to ensure the correct inference of its own small LLMs in order to reach the designated location safely. However, we still face the following challenges and difficulties.

First, enabling the LLM to generate highly interpretable UAV movement information is the primary challenge we aim to address. Although Wei et al [43] demonstrate that Chain-of-Thought (CoT) prompting can enhance LLMs' task understanding and accuracy, inferring UAV movement positions remains a highly complex task. This complexity stems from the need for the LLM to infer over dynamically changing information, including low-altitude obstacles, other UAVs, and moving boundaries. In our attempts with CoT prompting, the UAVs exhibited poor movement performance, and the LLM often failed to correctly output the UAVs' movement positions. In contrast, we have found that humans perform well when they see similar localized information of a scene because the brain performs multiple steps of thinking and reasoning, and after a series of eliminations and comparisons arrives at the current optimal position for the UAV to move to. Obviously, it requires training for LLMs to enable them to do so. Wang et al [41]. showed that fine-tuning some of the parameters of an LLM resulted in little change

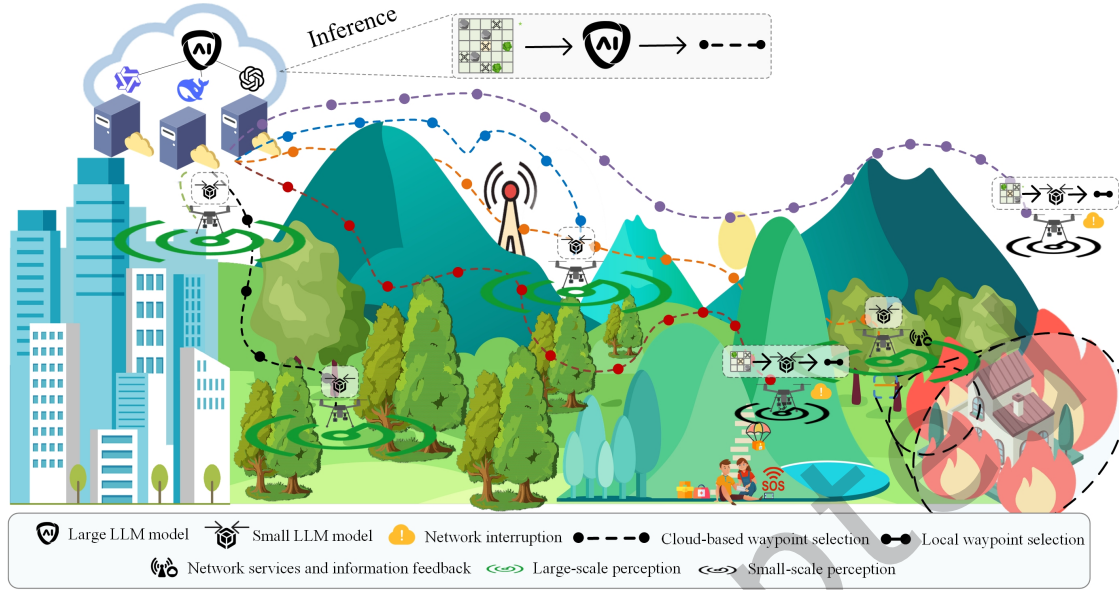


Fig. 1. Low-altitude UAV swarm rescue scenario.

in the performance of the downstream task compared to training an LLM from scratch, but the training time and resource consumption were greatly reduced. In addition, we need to carefully design prompt templates related to UAV path planning tasks and multi-stage deep reasoning. By incorporating multi-step reasoning processes for selecting optimal positions into these templates, we can dynamically generate fine-tuning datasets for the LLMs. To this end, we propose MSDTMD-SG to accomplish the above tasks. It first randomly generates UAV local observation information and embeds them into UAV path planning prompt template. Then, rewards are modeled for the positions in local observations using a reward mechanism similar to that in deep reinforcement learning (DRL). Next, a greedy search algorithm is utilized to search for the optimal moving position in the local observation. Finally, the optimal position and the reason why the other positions cannot be moved are dynamically embedded into the multi-step deep thinking template to generate the fine-tuning dataset to fine-tune the LLMs.

Second, Hao et al [14]. showed that during inference, LLMs may be misled by a small number of incorrect tokens, ultimately resulting in the wrong reasoning mindset. To address this problem, we designed the scene memory and replay learning mechanism. Specifically, when an erroneous inference occurs in the LLM resulting in damage to the UAV movement process, the scene information will be recorded, then scene replay and learning sample generation will be performed by MSDTMD-SG, finally the whole system will be capable of continuous learning through continuous fine-tuning of the LLMs. In summary, the main contributions of this paper are as follows:

- This study is the first to achieve the UAV swarm path planning task based on LLM in the edge computing scenario, and the movement decision is generated based on the local observation information of each UAV.
- We design LLM prompts and multi-step deep thinking templates for UAV path planning, and generate fine-tuning datasets for the LLM following human-like multi-step thinking and reasoning processes.
- Extensive experiments have been conducted to verify the effectiveness of the proposed method. The results demonstrate that our approach performs well in dynamic environments, achieves competitive system overhead, and exhibits strong adaptability along with robust self-learning capability.

The rest of the paper is organized as follows: Section 2 briefly reviews relevant works on EC, LLM, and UAV path planning in EC. Section 3 describes our system model and optimization objectives in detail. In Section 4, the algorithm MSDTMD-SG, our proposed dataset generation algorithm for UAV swarm path planning is presented. Next, Section 5 details the UAV swarm path planning approach and the proposed LLM continual learning mechanism. And then, in Section 6, we perform a variety of experiments and give the results and analysis. Finally, Section 7 concludes our work.

2 RELATED WORK

In this section, we first introduce the development and applications of edge computing (EC) architectures. Then, we explain how LLMs can progressively provide intelligent services efficiently in EC scenarios. Next, we review existing research on UAV path planning. Finally, we highlight the differences between our work and prior studies.

2.1 Edge Computing

The continuous development of 5G communication and Internet of Things (IoT) technology has brought the Internet of Everything (IoE) into reality [47], [25]. Under the condition of massive user device access, the near-user services provided by the EC architecture offer significant advantages in bandwidth, latency, and security compared to centralized cloud services. This has led to extensive research in areas such as vehicular networks, intelligent traffic scheduling, autonomous driving, and safety detection and early warning. Xiao et al. [46] designed a cloud-edge-vehicle hierarchical architecture for task offloading in vehicular networks. They proposed a vehicle mobility detection method and a DRL-based task offloading algorithm to achieve more efficient task offloading and resource allocation. Feng et al. [10] proposed a Vulnerability-Aware Task Scheduling (VATS) mechanism for intelligent transportation systems. By evaluating the vulnerability of edge intelligence entities, VATS enables dynamic allocation of secure tasks. In recent years, DRL has demonstrated significant advantages in intelligent transportation and autonomous driving, leading to a surge of innovative research in these fields [42], [9] [48]. Similarly, EC architecture plays an important role in security detection and early warning [12]. Nooh et al. [28] proposed an adaptive edge computing-based federated learning approach to enhance data processing efficiency and resource utilization in low-carbon sustainable environmental monitoring systems. Patel et al. [29] proposed a federated learning-based early warning mechanism for smart home environments to improve early detection and prediction of fire smoke. Although DRL algorithms are increasingly popular in highly dynamic and complex task environments, their black-box nature [38] makes them hard to interpret and limits their ability to recognize errors and self-optimize.

2.2 Large Language Model in EC

The introduction of the Transformer [37] marked a milestone in natural language processing (NLP) and has become the core architecture for most LLMs. In addition, LLMs are typically decoder-based models, such as GPT [54], LLaMA [32], DeepSeek [11], and Qwen [49]. In general, LLMs contain billions to tens of billions of parameters [16] and require extensive data and computational resources for prolonged training. To enable LLMs to operate more stably and efficiently in EC scenarios, researchers have conducted extensive studies on model lightweighting and compression. In studies such as [52], [34], and [56], researchers employed mixed-precision compression and approximate loss-aware techniques to reduce model size by up to 14.9× while maintaining performance comparable to the original model. Zeng et al. [53] proposed a novel arbitrary-bit quantization algorithm and inference framework called ABQ-LLM, which enables arbitrary-precision quantized inference on GPUs, significantly improving the inference speed and memory efficiency of LLMs. The above work makes it possible to deploy LLMs in edge scenarios and provide corresponding services, which accelerates the research on edge intelligent services. Zhao et al. [58] proposed an edge-end collaborative LLM deployment framework

that effectively reduces latency and energy consumption through a hybrid strategy of sequential and parallel token generation. Yao et al. [51] proposed a guided knowledge transfer (GKT) framework, enabling collaboration between large and small models without fine-tuning to generate responses, achieving a balance between efficient inference and high performance. Furthermore, Zhou et al. [60] were the first to investigate task offloading and resource allocation strategies based on LLMs in EC scenarios. Chen et al. [6] proposed an LLM-based cloud–edge collaborative autonomous driving system, which enhances the real-time performance and efficiency of edge systems. Overall, the application of LLMs in EC scenarios is still in the preliminary stage, the research scenarios are relatively simple, few studies have trained or fine-tuned LLMs for specific tasks, and LLMs’ foundational knowledge is not fully utilized.

2.3 UAV Path Planning in EC

In EC scenarios, UAVs primarily serve as mobile edge servers for task scheduling or assist cellular networks to provide network service in areas with poor communication quality. In [44], [45], researchers explored the trajectory planning and task scheduling problems of UAVs serving as mobile edge servers. Typically, UAV path planning during movement relies on heuristic or DRL-based methods. For example, Wei et al. [44] proposed a DRL-based algorithm to address trajectory planning and task scheduling for UAVs handling interdependent multi-task scenarios in complex environments. Liu et al. [24] employed a method combining ant colony optimization (ACO) with DRL to control the trajectory of UAVs. In addition, researchers have been continuously exploring UAV path selection in diverse edge computing scenarios. Studies [27], [55], and [33] investigated UAV path planning in various edge scenarios, including space–air–ground integrated networks, UAV-assisted vehicular network resource management, and edge environments with reconfigurable intelligent surfaces (RIS). In terms of collaborative UAV swarm services, Dhuheir et al. [8] divided tasks into multiple inference layers and employed a DRL-based approach to achieve path planning and distributed collaborative inference among UAVs. However, existing UAV path planning algorithms still face major challenges in terms of interpretability and adaptability to highly dynamic edge computing (EC) environments, which clearly cannot meet the growing demand for more intelligent services.

2.4 Differences with Existing Works

Existing work on LLMs in edge computing (EC) scenarios primarily focuses on simple cloud–edge collaborative deployments to provide fast and convenient inference services. Although there have been studies where LLMs guide autonomous vehicle movement, the scenario designs remain relatively simple. Existing UAV path planning algorithms, although they have achieved better performance, may suffer from cliff-like performance degradation when frequently faced with constantly dynamically changing environments, and this phenomenon is difficult to explain. In brief, the main differences between this study and existing work are as follows:

- 1) This study is the first to investigate LLM-based UAV swarm path planning in EC scenarios, where UAVs rely solely on local observation information to generate movement decisions.
- 2) We propose a human-like multi-step deep thinking sample generation algorithm to generate training samples for fine-tuning LLMs, enabling them to have better interpretability and environmental adaptability when dealing with UAV swarm path planning.
- 3) A continual learning mechanism with scene memory and replay is designed for LLMs to ensure the system recognizes its errors and actively corrects them.

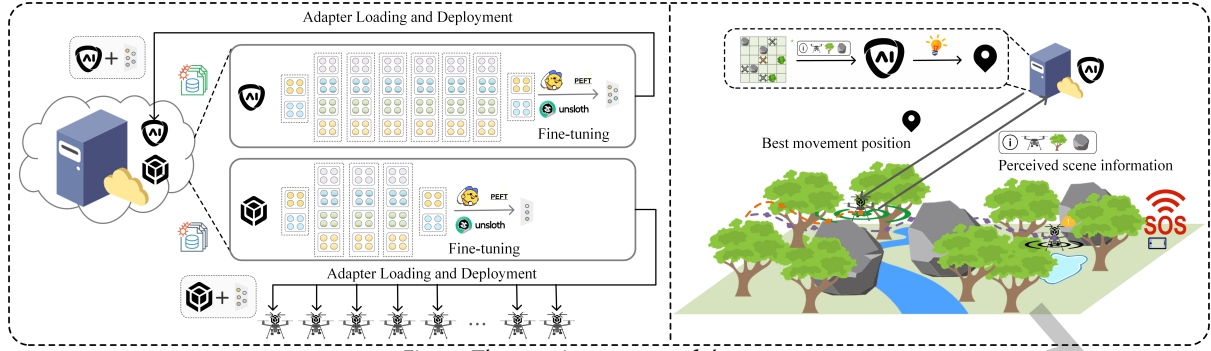


Fig. 2. The running process of the system.

3 SYSTEM MODEL

This section presents the system model of the proposed scheme. First, we introduce the UAV mobility control model, followed by the communication model between UAVs and the cloud server. Finally, we describe the LLM cost statistics model.

3.1 System Overview

We consider a UAV swarm rescue scenario, as illustrated in Fig. 1, where all UAVs depart from the starting point and travel to the rescue site to perform the tasks. Each UAV acts as an edge server, providing different types of rescue services at the rescue site. First, the left part of Fig. 2 illustrates the training and deployment process of LLMs before the mission begins. All LLMs are fine-tuned in the cloud using a UAV path planning dataset. The adapter parameters of the large LLM are combined with the base model and deployed in the cloud, while the adapter parameters of the small LLM are individually transmitted to each UAV and integrated with their onboard small LLMs for deployment. Next, the right part of Fig. 2 illustrates the process of UAVs moving from the starting point to the destination. When a UAV can communicate with the cloud, it will turn on its maximum range perception and upload the perceived information to the cloud, and the cloud-based large LLM then guides the UAV to travel longer distances and reach the destination as quickly as possible. Due to the possible existence of areas with poor communication quality, some UAVs may not be able to communicate with the cloud, at which point the UAVs will autonomously reduce their perceived range and use the smaller scope of perceived information to generate movement decisions. This strategy ensures the inference accuracy of their own small LLM model, so that they can reach the destination safely. In addition, the edge system contains m LLMs, where $m - 1$ are small LLMs and one is a large LLM deployed in the cloud. The set of LLMs is denoted as $M = \{M_1, M_2, \dots, M_{M-1}, M_M^{cld}\}$. The UAV swarm consists of N UAVs. Since each UAV is equipped with a small LLM, we have $N = M - 1$, and the set of UAVs is represented as $U = \{U_1, U_2, \dots, U_N\}$. The communication status between UAV k and the cloud is denoted by $\zeta_{k,cld}(t) \in \{0, 1\}$, where $\zeta_{k,cld}(t) = 1$ indicates that UAV k can communicate with the cloud at time step t , and $\zeta_{k,cld}(t) = 0$ otherwise.

3.2 UAV Control Model

Assume that the operation of the edge system can be described as T time steps, each time step lasting Δ seconds. $T = \{1, 2, \dots, t, \dots, T\}$, where the UAV swarm completes the path planning task in the T -th time step. We assume that all UAVs are at the same height H . For U_k , its coordinates at time step t can be expressed as $P_k(t) = (x_k(t), y_k(t), H)$. Since the initial position of each UAV is given and the height of the UAV is fixed, the optimization of the UAV waypoints mainly occurs in the xy -plane. Therefore, we can rephrase the coordinates of U_k as $P_k(t) = (x_k(t), y_k(t))$. At time step t , the velocity $V_k(t)$ and acceleration $a_k(t)$ of U_k can be expressed as:

$$V_k(t) = \frac{\|P_k(t+1) - P_k(t)\|}{\Delta}, \quad (1)$$

$$a_k(t) = \frac{V_k(t+1) - V_k(t)}{\Delta}, \quad (2)$$

and $P_k(t+1)$ can be expressed as:

$$P_k(t+1) = (x_k(t) + V_k(t)\Delta \cos \theta_k(t), y_k(t) + V_k(t)\Delta \sin \theta_k(t)), \quad (3)$$

where $\theta_k(t) \in [-\pi, \pi]$ represents the heading angle of UAV k at time step t . $\theta_k(t)$ can be denoted as:

$$\theta_k(t) = \arctan_2(x_k(t+1) - x_k(t), y_k(t+1) - y_k(t)). \quad (4)$$

Additionally, the velocity of UAV k at time step t needs to satisfy the condition $V_{min} \leq V_k(t) \leq V_{max}$.

The UAVs in the system can harvest energy from external sources to charge their batteries for continuous operation. The energy harvested by U_k at time step t , denoted as $E_k^{coll}(t)$, which can be expressed as follows according to [22]:

$$E_k^{coll}(t) = a_k^{solar} b^{solar} c_k \exp\left(\frac{-\kappa}{\cos q^{solar}(t)} (1 - 2.2556 \times 10^{-5} H)^{5.2561}\right), \quad (5)$$

$c_k \in (0, 1)$ represents the charging efficiency of the U_k 's solar panel, a_k^{solar} is the size of the U_k 's solar panel, and b^{solar} is the constant power intensity of the solar beams. κ is the total atmospheric extinction. $q^{solar}(t)$ denotes the solar zenith angle at time step t , and H is the U_k 's altitude.

3.3 Communication Model

The communication between the UAV and the cloud consists of two stages: (1) transmitting surrounding perception information to the cloud for path planning guidance, and (2) receiving movement instructions inferred by the large LLM deployed on the cloud. Since the data size of the information returned from the cloud to the UAV is negligible, this paper primarily focuses on the communication overhead in the first stage.

The UAVs in the system communicate with cloud via wireless channels. According to [17] and the 3GPP technical report [1], the path loss between U_k and cloud at time step t is given by:

$$h_{k,cld}(t) = \frac{\epsilon_{ref,cld}}{(x_{cld} - x_k(t))^2 + (y_{cld} - y_k(t))^2 + H^2}, \quad (6)$$

where $\epsilon_{ref,cld} = 10^{-5.84}$ represents the received power at the reference distance of 1 m.

At time step t , the uplink rate between U_k and cloud can be denoted as:

$$R_{k,cld}(t) = B_{k,cld} \log_2\left(1 + \frac{P_k^{pw} h_{k,cld}(t)}{\sigma^2}\right), \quad (7)$$

where $B_{k,cld}$ is the transmission bandwidth between U_k and cloud, P_k^{pw} is the transmission power of U_k , and σ is the noise power. The data size of the perceived environmental information by U_k at time step t is denoted as $D_k(t)$. Then, the communication delay $\mathcal{T}_{k,cld}^{up}(t)$ between U_k and cloud at time step t can be expressed as:

$$\mathcal{T}_{k,cld}^{up}(t) = \frac{D_k(t)}{R_{k,cld}(t)} \zeta_{k,cld}(t). \quad (8)$$

Meanwhile, the uplink communication energy consumption $E_{k,cld}^{up}(t)$ between U_k and cloud can be expressed as:

$$E_{k,cld}^{up}(t) = \mathcal{T}_{k,cld}^{up}(t) P_k^{pw} \zeta_{k,cld}(t). \quad (9)$$

3.4 LLM Inference Model

The inference cost of LLM primarily consists of two aspects: (1) The time required to generate a token sequence based on the given input prompt, and (2) the computational consumption required for token sequence generation. The following provides a detailed explanation of these two aspects.

Firstly, regarding the time cost of LLM inference, Zhou et al. [60] highlight that the input to the LLM is split into smaller tokens, and then the LLM uses these tokens as predictors for the next token through autoregressive decoding. Newly generated tokens are appended to the input sequence iteratively, serving as predictors for subsequent tokens until the generation process is complete. Therefore, the time cost of the LLM inference process can be expressed as the sum of the time required to generate the first token and the time required to generate each subsequent token. For prompt i , the inference time cost of LLM m , denoted as $\mathcal{T}_{m,i}^{gen}$, can be given by:

$$\mathcal{T}_{m,i}^{gen} = (t_m^{TTFT} + n_i^{token} t_m^{TPOT}) \vartheta_{m,i}(t), \quad (10)$$

where $\vartheta_{m,i} \in \{0, 1\}$ indicates whether LLM m receives and processes prompt i . A value of 1 means that prompt i is received and processed, while 0 means it is not. t_m^{TTFT} and t_m^{TPOT} represent the time required for LLM m to generate the first token and each subsequent token, respectively. n_i^{token} denotes the number of tokens generated for prompt i . Researchers can easily test and obtain the values of t_m^{TTFT} and t_m^{TPOT} from industry-standard platforms to evaluate different LLMs [3]. By using TTFT and TPOT metrics reported on standardized platforms, we can ensure the reproducibility of the experimental while eliminating potential deviations caused by hardware differences.

Secondly, the computational consumption during LLM inference can be measured using computational complexity, which is typically represented by the number of floating-point operations per second (FLOPs). Consider an LLM with κ attention heads and F layers, where d_{in} and d_{mid} denote the input sequence length and hidden dimension, respectively. The computational complexity of the attention mechanism and feed-forward network in the f -th layer can be expressed as $FLOPs(f) = \frac{3d_{in}d_{mid}^2}{\kappa} + \frac{2d_{mid}^2d_{mid}}{\kappa} + 9d_{in}d_{mid}^2$. For prompt i , the computational cost of LLM m , denoted as $E_{m,i}^{gen}$, can be expressed as:

$$E_{m,i}^{gen} = \sum_{f=1}^F FLOPs(f) \vartheta_{m,i}(t). \quad (11)$$

3.5 Formulation of the Problem

The research objective of this paper is to find the optimal path for each UAV in the UAV swarm to move to the destination, it means that the system operating time T and system computational consumption E_{all} required for the UAV swarm to reach the destination are minimized. Meanwhile, if each UAV in the system reaches the destination while ensuring minimal computational consumption and system operating time, it indicates that the UAVs have followed the optimal movement paths to their destinations, and the two are equivalent. The communication and LLM reasoning cost in the system can be expressed as:

$$C_{com} = \sum_{t=1}^T \sum_{k=1}^N (T_{k,clid}^{up}(t) + E_{k,clid}^{up}(t)). \quad (12)$$

$$C_{LLM} = \sum_{t=1}^T \sum_{m=1}^M \sum_{i=1}^N (\mathcal{T}_{m,i}^{gen} + E_{m,i}^{gen}). \quad (13)$$

Thus, the original problem can be formulated as an optimization problem $\mathcal{P}$, which aims to minimize system computational cost and operation time. It can be expressed as:

$$\mathcal{P} = \min(C_{com} + C_{LLM}). \quad (14)$$

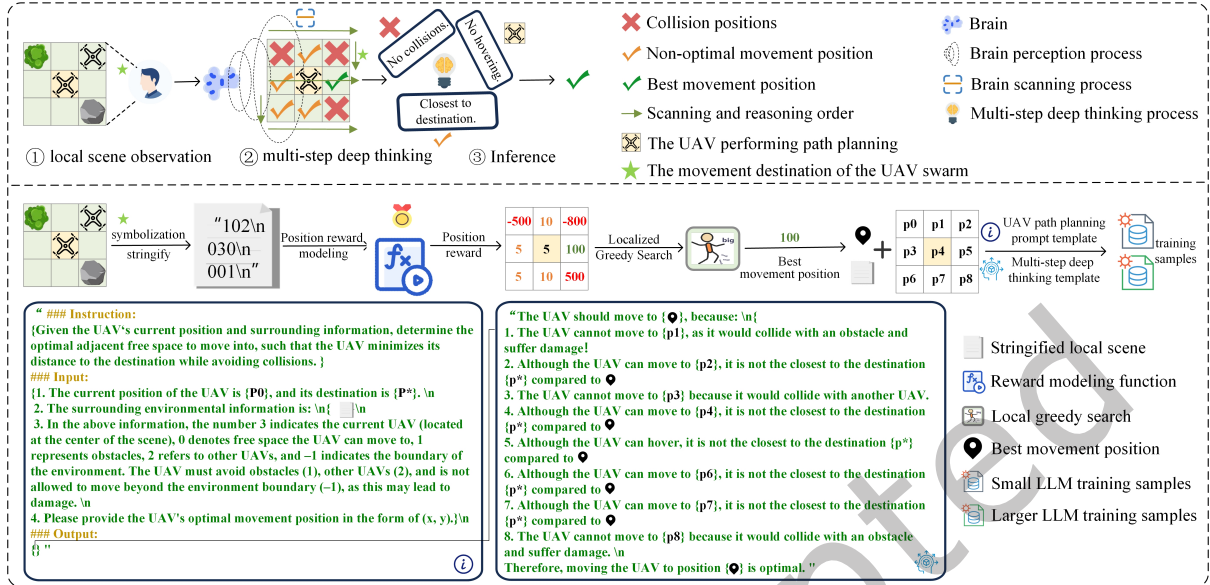


Fig. 3. The process of human multi-step deep thinking and the generation of fine-tuning samples for LLMs.

$$s.t. \quad C1 : V_{min} \leq V_k(t) \leq V_{max}, \forall k \in \mathcal{N} \quad (14a)$$

$$C2 : \sum_{k=1}^N B_{k,cld} \leq B_{max} \quad (14b)$$

$$C3 : \sum_{k=1}^N p_k^{pw} \leq P_{max}, \quad (14c)$$

where constraint C1 restricts the speed of the UAV must be between the minimum speed V_{min} and the maximum speed V_{max} . Constraints C2 and C3 mean that the total bandwidth allocated to all UAVs does not exceed the system's maximum bandwidth B_{max} , and the sum of the transmission power of all UAVs does not exceed the maximum transmission power P_{max} allocated by the EC system.

4 MULTI-STEP DEEP THINKING MOVEMENT DECISION SAMPLES GENERATION ALGORITHM

This section introduces how to enable the LLMs to perform human-like multi-stage deep reasoning, which includes two parts: (1) the generation of multi-step deep thinking movement decision samples through our proposed multi-step deep thinking movement decision samples generation algorithm (MSDTMD-SG), and (2) the fine-tuning of LLMs for multi-step deep reasoning.

4.1 Insights

For humans, determining the optimal movement position of a UAV based on its local perception information typically involves a multi-step reasoning process. First, we subconsciously scan the UAV's perceived environment, then use our brain to identify obstacles, other UAVs, and scene boundaries. Next, we exclude these locations from the set of movable positions. Finally, we determine the UAV's optimal movement position by selecting the position closest to the destination from the remaining movable positions.

Based on the above analysis, applying LLMs to guide UAV swarm path planning presents the following challenges: (1) how to enable the LLMs to identify immovable positions; (2) how to enable the LLMs to recognize which movable position is closest to the destination; and (3) how to train the LLMs to learn the human-like multi-step reasoning process.

For challenge (1), we can perform symbolic modeling of the perceived environment by using different textual symbols to distinguish obstacles, other UAVs, scene boundaries, and the UAV itself. Inspired by the reward mechanism in deep reinforcement learning, we can construct a reward model to assign a reward value to each position in the perceived environment, where immovable positions receive high movement penalties, while the rewards of movable positions increase as their distance to the destination decreases. Therefore, the optimal position in challenge (2) is the movable point with the highest reward value. Finally, for challenge (3), since the perceived scene provides only partial information about the entire environment, we can apply greedy search on the perceived scene after reward modeling to determine the optimal moving point. To enable the LLMs to develop human-like reasoning abilities, we design a dedicated multi-step deep thinking template and generate multi-stage deep thinking samples based on the optimal movement point for fine-tuning the LLMs.

4.2 The Process of Multi-step Deep Thinking Samples Construction

The construction of the multi-step deep thinking process is the core of the proposed MSDTMD-SG, which consists of several steps, and the details of these steps are described below.

a) Symbolic Modeling of Local Perception: Cameras, radars, and other sensors onboard the UAVs can perceive their local surroundings, and the perceived information can be transformed into textual descriptions through multimodal techniques [50]. This work focuses on how to transform these textual descriptions into symbolic representations that are easier for LLMs to understand and process. For the perceived information $S_k(t)$ perceived by UAV k at time step t , we symbolize it as:

$$S_k(t) = [s_{ij}(t)]_{n_k(t) \times n_k(t)}, \forall i, j \in \{1, \dots, n_k(t)\}, \quad (15)$$

where $n_k(t)$ is the perceived diameter of the UAV k at time step t , $s_{ij}(t) \in \{-1, 0, 1, 2, 3\}$, -1 represents the environment boundary, 1 denotes obstacles, 2 refers to other UAVs in the scene, and 3 indicates the current UAV. In addition, the current UAV is located at the center of the $S_k(t)$, i.e., the center element of $S_k(t)$ is $a_{\lceil n/2 \rceil, \lceil n/2 \rceil} = 3$.

b) Reward Modeling of Local Perception: After symbolizing the local perception environment, it is necessary to enable the LLMs to understand the real-world meaning of each symbol and distinguish between movable and immovable positions. Inspired by the reward function in DRL, we model rewards for each position in the perceived scene, giving greater penalties to immovable positions, and higher rewards to movable positions that are closer to the destination. The specific reward functions are as follows:

First, $C_k^{mv}(t)$ denotes the reward function for UAV k when it successfully moves without collision at time step t . $C_k^{mv}(t)$ encourages the UAV to move towards the destination rather than hovering. Moreover, $C_k^{mv}(t)$ gradually decreases as the UAV's total running time increases.

$$C_k^{mv}(t) = \frac{1}{T_k^{run}(t)} \left(\frac{d_k(t)}{n_k(t)} + \frac{d_k^{ori} - \tilde{d}_k^{tar}(t)}{d_k^{ori}} \right) R_{mv} d_k(t), \quad (16)$$

where $T_k^{run}(t)$ represents the running time of UAV k , $d_k(t)$ denotes its movement distance at time step t , d_k^{ori} is the initial distance between UAV k and its destination, $\tilde{d}_k^{tar}(t)$ is the distance from the current position to the destination, and R_{mv} is a constant representing the movement reward factor.

Second, when the UAV reaches its destination, we need to give it a large reward for task completion. Specifically the shorter the time the UAV takes to reach its destination, the bigger the reward, we use $C_k^{fin}(t)$ to represent this

reward function, R_{fin} represents the reward factor for the UAV to reach its destination, $C_k^{fin}(t)$ can be given by:

$$C_k^{fin}(t) = \frac{1}{T_k^{run}(t)} R_{fin}. \quad (17)$$

Third, when a UAV collides with obstacles or other UAVs in the environment, a large penalty should be assigned to indicate that such positions are immovable. The penalty at time step t for UAV k in case of a collision is given by the reward function $C_k^{brk}(t)$, which can be represented as:

$$C_k^{brk}(t) = T_k^{run}(t)(R_{brk} + R_{uav} \mathbb{N}_k^{uav}(t)), \quad (18)$$

where R_{brk} and R_{uav} denote the penalty factors for collisions with obstacles and other UAVs, respectively. $\mathbb{N}_k^{uav}(t)$ represents the number of UAVs that UAV k collides with at time step t .

Finally, the penalty for a UAV flying out of the environmental boundary is given by the reward function $C_k^{oor}(t)$, which can be denoted as:

$$C_k^{oor}(t) = T_k^{run} R_{oor} \frac{1}{d_k(t)}, \quad (19)$$

where R_{oor} denotes the penalty factor for UAVs flying out of the environmental boundary.

c) Greedy Search for Optimal Movement Position: Since the perceived scene of the UAV is localized, we can easily use greedy search to find out the position with the maximum reward, i.e., the optimal moving position of the UAV at time step t , $P(t+1)$. For UAV k , P_k^s denotes the positions of the scene perceived by UAV k . The greedy search process to obtain $P_k(t+1)$ can be expressed as:

$$P_k(t+1) = \arg \max_{P_k^s \in P_k^s} (C_k^{mv}(t) + C_k^{fin}(t) + C_k^{brk}(t) + C_k^{oor}(t) | p_k^*), \quad (20)$$

c) Multi-step Deep Thinking Sample Generation: Even with optimal movement positions, LLMs still do not have a good understanding of each element in the scene, and it is hard to scan and determine the optimal movement position step-by-step for the perceived scene after reward modeling. This is because LLMs need to recognize the elements in the scene based on the input prompt and answer how the UAV should move in the current perceived scene. In addition, due to the need to evaluate multiple positions within the scene, directly obtaining the optimal position from LLMs compromises interpretability and fails to provide a clear reasoning process. To enable LLMs to reason like humans through multi-stage deep thinking, we design a multi-stage deep thinking template. This template is then used to generate deep thinking samples for LLMs, which are utilized for fine-tuning and reasoning learning. Specifically, the sample generation function is denoted as G_{smp} , which can be expressed as:

$$G_{smp} = W_{pmt}(S_k(t), P_k(t), P_k^{tar}) + W_{dth}(S_k(t), P_k^s, P_k(t+1)), \quad (21)$$

where W_{pmt} denotes the UAV path planning prompt template and W_{dth} represents the multi-stage deep thinking template. Fig. 3 illustrates the process of human multi-step deep thinking and the generation of fine-tuning samples for LLMs. Additionally, we provide flexible output configurations for the LLM. The multi-stage reasoning process shown in Fig. 3 is only output during LLM fine-tuning to enable them to learn human-like reasoning patterns. During the inference stage, the LLM only outputs the UAV's movement decisions.

4.3 Multi-step Deep Thinking Learning for LLMs

For LLMs, the most important thing to do to control the path movement of the UAV is to learn the deep thinking pattern in the training samples. Compared to retraining the entire LLM, fine-tuning allows it to stably learn and adapt to downstream tasks with lower computational cost by leveraging its existing knowledge and language capabilities [41]. In this paper, we use Unsloth AI's open source fine-tuning framework Unsloth to fine-tune LLMs [2]. Through stochastic environment generation and multiple calls to Eq. 21, we can use a large number of samples to fine-tune the LLMs so that they can learn multi-stage deep thinking and infer the optimal movement position of the UAV. The cross-entropy loss is used as the loss function for fine-tuning the LLMs, which can be formulated as:

$$\mathcal{L} = -\frac{1}{\sum_{l=1}^{L^{gen}} m_l} \sum_{l=1}^{L^{gen}} m_l \cdot \log \hat{y}_l[y_l], \quad (22)$$

Where L^{gen} is the number of tokens generated by LLM, m_l is an indicator function to point out whether the generated text $\hat{y}_l$ belongs to the input prompt words of LLM or not, with the value of 0 if yes, and the value of 1 if no, and y_l is the text in the training sample.

4.4 Description of MSDTMD-SG

Algorithm 1 provides a detailed description of the MSDTMD-SG. It first needs to initialize the sample set $\mathcal{D}_{smp}$, the number of generated samples num , and the size of the environment, L (line1). Then, for each sample generation process, the current position of the UAV which is denoted as P and its destination P^{tar} are randomly generated (lines 3-4). Next, `RandomInitScene( $\cdot$ )` is used to randomly generate a local perception scene of size $n \times n$ and return the positions within the scene (line 5). After that, the reward function is used to model each position in the scene, and a greedy search is employed to find the optimal movement position P^{bst} (lines 6-7). Next, the UAV path planning prompt template W_{pmt} and the multi-step deep thinking template W_{dth} are applied to the relevant information to generate the input prompt and the multi-step deep thinking process, respectively. These are then concatenated and added to $\mathcal{D}_{smp}$ (lines 8-11). Finally, once the number of generated deep thinking samples reaches N_{smp} , the process terminates and returns $\mathcal{D}_{smp}$.

5 UAV SWARM PATH PLANING BASED ON LLM USING LOCAL ENVIRONMENTAL OBSERVATIONS

This section introduces how UAVs in a swarm collaborate with the cloud to complete path planning tasks. We first describe how UAVs operate under different channel conditions. Then, we present LLMs' scene memory and learning mechanism.

5.1 UAV Swarm Path Planning

For each UAV in the swarm, the task is to reach the destination without collisions or flying out of the boundaries. First, the motion path of UAV k can be represented by a series of waypoints, i.e., $L_k^{pth} = \{P_k^{sta}, P_k(1), \dots, P_k(t), P_k^{end}\}$, where P_k^{sta} represents the start point of UAV k and P_k^{end} represents the end point of its movement. Second, when UAV k completes its task, there is $P_k^{end} = P_k^{tar}$, otherwise it does not hold. Finally, communication conditions also affect the UAV's movement decisions, when communication conditions are good, the UAV k will turn on its maximum range perception and upload the perceived information to the cloud. Then, UAV k is guided to move greater distances by large LLMs deployed in the cloud. In contrast, in poor communication conditions, the UAV k will autonomously reduce its perceived range and use the smaller scope of perceived information to generate movement decisions by using its own deployed small LLM model. At time step t , the moving decision process under both conditions can be expressed as follows:

Algorithm 1: Multi-step Deep Thinking Movement Decision Samples Generation Algorithm

Input: $\mathbb{N}_{smp}$: The number of samples
 n : The UAV's perceived diameter
 W_{pmt} : The input prompt template for LLM
 W_{dth} : The multi-stage deep thinking template

Output: $\mathcal{D}_{smp}$: The set of generated samples

- 1: Initialize $\mathcal{D}_{smp}$ to empty set, the number of generated samples $num = 0$, the scene size L
- 2: **while** $num \leq \mathbb{N}_{smp}$ **do**
- 3: $P \leftarrow \text{UniformRandom}(0, L, size = 2)$
- 4: $P^{tar} \leftarrow \text{UniformRandom}(0, L, size = 2)$
- 5: $S, P^s \leftarrow \text{RandomInitScene}(n, L, P)$
- 6: $C \triangleq C^{mv} + C^{fin} + C^{brk} + C^{ovr}$
- 7: $p^{bst} \leftarrow \arg \max_{P^s \in P^s} (C|p^*)$
- 8: $\mathbb{T}^{pmt} \leftarrow \text{StrFmt}(s = W_{pmt}, S, P, P^{tar})$
- 9: $\mathbb{T}^{dth} \leftarrow \text{StrFmt}(s = W_{dth}, S, P^s, p^{bst})$
- 10: $sample \leftarrow \text{StrConcat}(\mathbb{T}^{pmt}, \mathbb{T}^{dth})$
- 11: $\mathcal{D}_{smp} \leftarrow \text{AddElement}(\mathcal{D}_{smp}, sample)$
- 12: $num \leftarrow num + 1$
- 13: **return** $\mathcal{D}_{smp}$

$$P_k^{con}(t+1) = LLM_{cld}(P_k^{pmt}(t), \theta_{cld}(t)). \quad (23)$$

$$P_k(t+1) = LLM_k(P_k^{pmt}(t), \theta_k(t)). \quad (24)$$

In highly dynamic environments, it is difficult to guarantee that all UAVs can successfully complete their path planning tasks. However, we expect these UAVs to become increasingly intelligent after failures—like students, they should be able to recognize and correct their mistakes. To address this issue, we design a scene memory and replay learning mechanism for LLMs deployed at different ends. A detailed description of this mechanism is provided in Section 5.2.

5.2 Scene Memory and Replay Learning Mechanism

We hope that LLMs are like students, allowed to make mistakes and have a lack of understanding, but we expect that they can recognize their mistakes and learn to correct them. Although LLMs will reason step by step and obtain the UAV's moving position, the LLMs may have a wrong thought orientation during token generation, i.e., a certain token may change the inference direction of the LLM's thought, which leads to a wrong reasoning result. Therefore, it is necessary to establish an LLM-based automated continual learning and reasoning enhancement mechanism to recognize errors and continuously improve the thinking ability.

To reflect on errors we need to memorize them, and since LLMs on the cloud and UAVs will use different ranges of perceptual information for reasoning, we store different memories for the cloud and the UAVs and denote them by C_{cld}^{cah} and C_k^{cah} , respectively, they can be denoted as follows:

$$C_{cld}^{cah} = \{(P_k(j), \tilde{S}_k(j), P_k^{tar}) | 1 \leq j \leq T, 1 \leq k \leq \mathcal{N}\}, \quad (25)$$

$$C_k^{cah} = \{(P_k(j), S_k(j), P_k^{tar}) | 1 \leq j \leq T, 1 \leq k \leq \mathcal{N}\}, \quad (26)$$

Table 1. Simulation Parameters

Parameters	Value
Noise power, σ^2	-110 dBm
Minimum UAV speed, V_{min}	0 m/s
Maximum UAV speed, V_{max}	5.65 m/s
Bandwidth $B_{k,cl}$	[2, 10] MHz
Transmission power p_k^{pw}	[100, 200] mW
Total bandwidth B_{max}	100 MHz
Total power P_{max}	2000 mW
Time step duration Δt	0.5 s

where j denotes the time step at which the LLM inference error occurs, and $\tilde{S}_k(j)$ denotes the perceived information of UAV k at time step j when it can communicate with the server.

Next, the proposed MSDTMD-SG will be used to perform scene replay on the information in the memory and learn how to reason correctly. The cloud-based LLM will fine-tune and learn using the C_{cl}^{cah} uploaded by the UAVs. To avoid training on the UAVs, the small model of the UAV will be backed up in the cloud. All uploaded C_k^{cah} will be used to fine-tune the small LLM. After the fine-tuning, the small LLM is updated on the cloud and then the adapter parameters are distributed to each UAV.

6 EXPERIMENT

In this section, we conduct extensive experiments to evaluate and analyze the proposed method. Specifically, we first compared the path planning performance of various large and small LLMs after fine-tuning for UAV swarm path planning, and selected the best-performing large and small LLMs as the baseline models for our proposed method. Then, we compare the path planning performance and system consumption of the proposed method with the comparison methods. Finally, we conducted experiments to demonstrate the effectiveness of the proposed scene replay and learning mechanism.

6.1 Experiment settings

Simulation settings. We simplify the EC rescue scenario as a $20m \times 20m$ plane, which is divided into $1m \times 1m$ square grids. The UAV swarm consists of 10 UAVs, and their initial positions are randomly initialized and follow a discrete uniform distribution. We set 8% of the grids as obstacles and place the rescue point (destination) at the top-right corner of the scenario. The cloud server's location is randomly assigned within the scenario and does not change over time. Additionally, low-altitude UAVs typically fly below 120m [5]. Since we assume the UAVs need to perform tasks such as item delivery and on-site data transmission, requiring near-ground operation, we set the UAV flight altitude H to 20m. The settings of other variables in the system refer to the works [30] and [26], with the variable values listed in Table 1. In our experiments, we considered a maximum model size of 8B, as we found that the fine-tuning samples generated by the proposed method enable the LLM to achieve strong path planning performance even at a relatively small parameter scale.

All experiments were conducted on our lab server equipped with an Intel Core i7-14700K CPU and two NVIDIA GeForce RTX 4090 GPUs. The experimental code is based on Python 3.10. The version of the Transformers library used for pre-trained models is 4.49.0, the high-performance computing library Triton is version 3.0.0, and the LLM fine-tuning library Unsloth is version 2025.2.15. Additionally, all models and fine-tuning code in this paper are implemented based on PyTorch 2.4.1. We set the temperature of all LLMs to 0.9. During the LoRA [15] fine-tuning process with Unsloth, the rank of the low-rank matrices is set to 128, and the scaling factor LoRA alpha is set to 16. In addition, our method provides an inference paradigm for UAV swarm path planning, which is dynamically embedded into the generated samples through the proposed MSDTMD-SG. Fine-tuning strategies

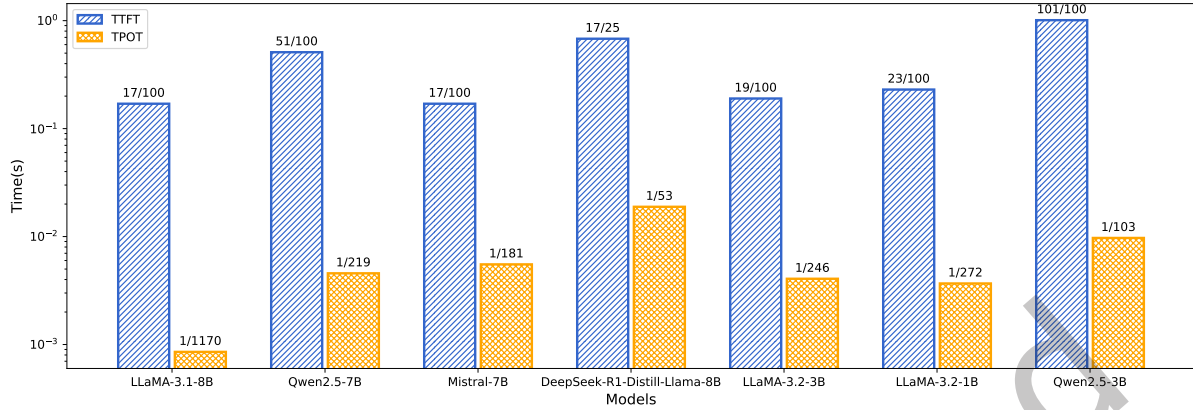


Fig. 4. The TTFT and TPOT of different LLM models.

only influence how large language models learn these inference patterns, which is not the core focus of this paper. Therefore, this paper only considers Unsloth’s supervised fine-tuning method.

Datasets in experiment. The LLM fine-tuning datasets at different ends of the experiment are generated by the proposed MSDTMD-SG algorithm. The obstacles in the UAV observation scene, the locations of other UAVs as well as the destinations are randomly generated and the generation process follows a discrete uniform distribution. The fine-tuning dataset contains a total of 20,000 training samples. To compare the movement decision performance of different methods, we additionally generated 5,000 evaluation samples. Meanwhile, during the sample generation process, the scenario size was set to $100m \times 100m$ to enable more comprehensive generation of local information. Note that since UAVs perform path planning based on local observations, the scenario size does not affect the algorithm’s performance. To intuitively and efficiently demonstrate the effectiveness of the proposed method, the scenario size in the experiments was set to $20m \times 20m$.

Baselines. To demonstrate the effectiveness of the proposed method, we conducted comparisons with both heuristic-based and DRL-based path planning algorithms, as detailed below:

1) DRL-based UAV swarm path planning (DRL-USPP): DRL-based methods have been widely applied to path planning tasks [4], [57]. Since the action choices of UAVs in a swarm are discrete, and DRL algorithms such as AC [39] and DDPG [59], which are designed for continuous action spaces, perform poorly in such discrete settings, we adopt a DQN [61] based path planning method as a baseline for comparison in this work.

2) ACO-based UAV swarm path planning (ACO-USPP): Among heuristic algorithms, the Ant Colony Optimization (ACO) [18] algorithm is well-suited for varying environments due to its robustness to parameter fluctuations and initial conditions. It has been widely used to solve classical NP-hard problems such as the Traveling Salesman Problem. The use of pheromones and heuristic distance functions enables ACO to achieve good performance without complex training procedures. Therefore, in this study, we adopt the ACO-based path planning method as a baseline for comparison.

6.2 Performance Comparison of Different LLMs

Different LLM models may exhibit performance variations due to differences in their architectures. Therefore, we first compare the path planning performance of various LLM models. We dynamically generated 20 scenarios and evaluated the performance based on task completion rate, which measures how well the 10 UAVs in the swarm reach the destination. Note that large and small LLM models are compared separately. Next, we selected the best-performing large LLM and small LLM models as the foundation models for the cloud and edge ends for subsequent experiments. Finally, we evaluated the system overhead of three schemes: using only the large LLM

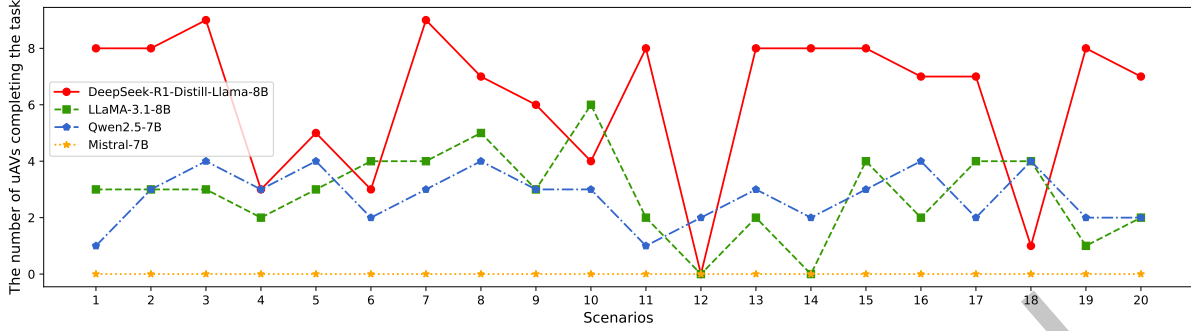


Fig. 5. The comparison of path planning performance of large LLM models.

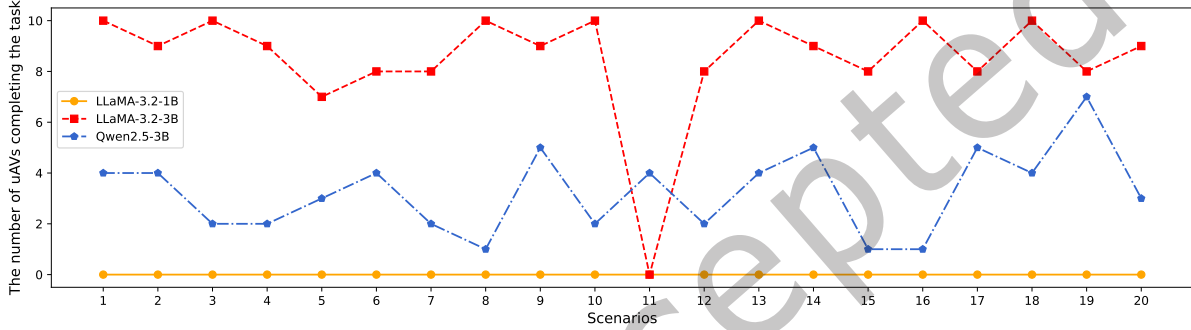


Fig. 6. The comparison of path planning performance of small LLM models.

deployed on the cloud, using only the small LLM deployed on the edge, and a hybrid approach where only a subset of UAVs use the large LLM deployed on the cloud.

In the path planning performance comparison of large LLM models, we evaluated LLaMA-3.1-8B, Qwen2.5-7B, Mistral-7B [19], and DeepSeek-R1-Distill-Llama-8B. In the path planning performance comparison of small LLM models, we evaluated quantized versions of LLaMA-3.2-3B, LLaMA-3.2-1B, and Qwen2.5-3B, all quantized to 4-bit precision. Fig. 4 illustrates the TTFT and TPFT of different LLM models. Except for LLaMA-3.1-8B, which shows a notable advantage in TPFT compared to other models, the remaining models exhibit relatively close performance in both TTFT and TPFT. Fig. 5 and Fig. 6 show the path planning performance comparison between large LLMs and small LLMs. It is evident that DeepSeek-R1-Distill-Llama-8B outperforms all other large models. LLaMA-3.1-8B has a slight advantage over Qwen2.5-7B. Meanwhile, LLaMA-3.1-8B does not perform as well as DeepSeek-R1-Distill-Llama-8B, suggesting that even if the models have the same number of parameters, the way in which the base model is trained can have an impact on the model's subsequent learning ability. However, surprisingly the fine-tuned Mistral-7B performs very poorly, which on the one hand highlights the complexity of the UAV swarm path planning task, and on the other hand confirms the existence of a performance gap between different LLM models [7]. In addition, we can find that LLaMA-3.2-3B has the best performance in the comparison of the path planning performance of small LLMs. However, LLaMA-3.2-1B had the worst performance with no UAV completing the task, indicating that the parameters of the model are too small to support the completion of a complex path planning task. Finally, LLaMA-3.2-3B outperforms Qwen2.5-3B for the same four-bit quantization again indicating that there are performance differences between different LLM models.

Based on the above comparison of different LLMs, we select DeepSeek-R1-Distill-Llama-8B and LLaMA-3.2-3B as the cloud-side and edge-side foundation models, respectively, for subsequent experiments. Realistic scenarios usually have areas with poor communication status, so we set up three LLM-based path planning schemes to

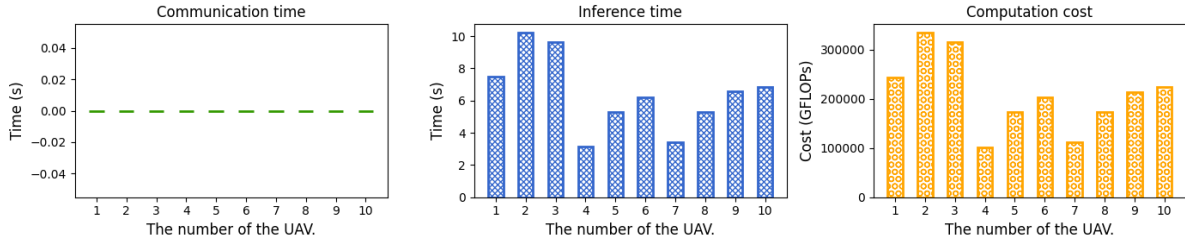


Fig. 7. The system cost using only small LLMs.

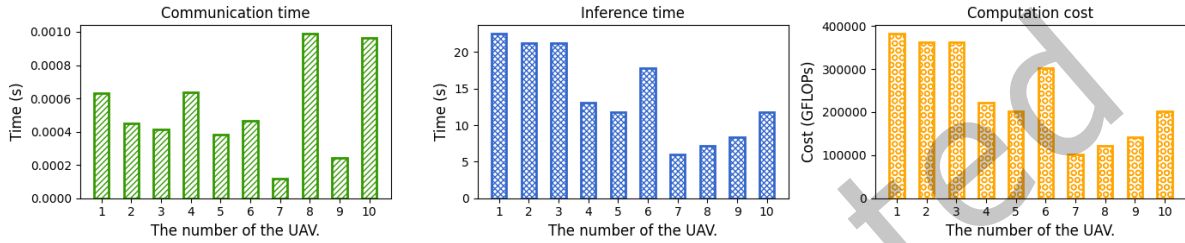


Fig. 8. The system cost using only large LLMs.

correspond to the cases of good communication status, good communication status of some UAVs, and very poor communication status, they are large LLM only, hybrid LLM, and small LLM only. Fig. 7, 8, and 9 show the experimental results of different LLM schemes. Specifically, we fixed the EC scenario, set the number of UAVs to 10, and uniformly compared the communication time, inference time, and computational cost of different schemes. It can be seen that the scheme of small LLM only, has lower reasoning time and computational consumption with no communication delay, which is due to the fact that the UAVs in the UAV swarm in the case of very poor communication state will have smaller consumption as they only rely on their own small LLMs for their movement decisions. In contrast, the scheme of the large LLM only not only consumes more inference and computation, but also increases the communication time. However, although we measured these costs, in actual operation only communication latency needs to be considered. The experimental results using the hybrid LLM scheme also corroborate our analysis above, and it can be seen that some UAVs have no communication latency and relatively low inference and computation consumption, which is caused by their inability to communicate with the cloud and use their own small LLMs. Finally, we can see that the consumption of different UAVs in the three LLM-based schemes is highly fluctuating, due to the fact that the UAVs may have to adjust new paths (temporary obstacle avoidance or bypassing the closer UAVs) due to the change of local information, resulting in the time to reach the destination being different for different UAVs.

In order to compare the system consumption of different LLM schemes, we conducted experiments and compared the total running time, computational consumption, and the decision steps used for all UAVs to complete the path planning task for the different schemes. As shown in Fig. 10, the small LLM only scheme has lower system runtime and computational consumption, but the number of decision steps required for all UAVs to complete the path planning is larger than the other two schemes, it due to the fact that the large LLM only scheme uses a larger range of localized observations at decision step, and can move a greater distance. Additionally, we can observe that the hybrid LLM scheme performs comparably to the other two methods.

6.3 Performance Comparison of Different Methods

To compare the performance differences between LLM-based methods and traditional path planning algorithms, we fixed the EC scenario and evaluated LLM-based methods against DRL-USPP and ACO-USPP. The experimental

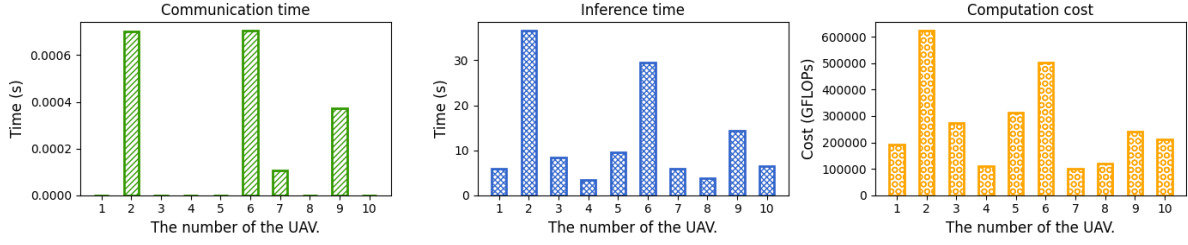


Fig. 9. The system cost using hybrid LLMs.

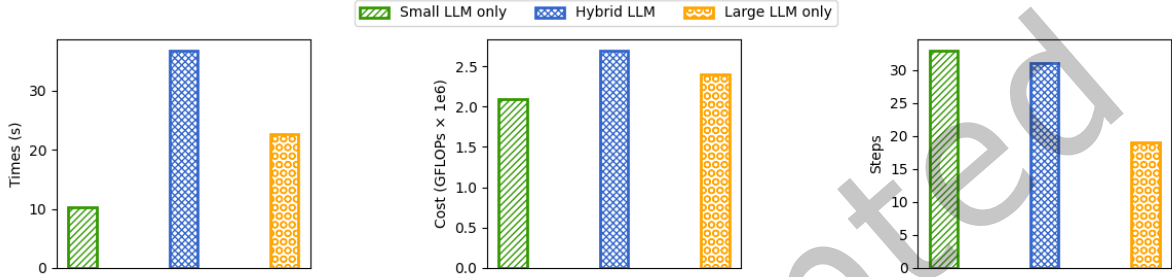


Fig. 10. The comparison of the performance of path planning based on LLM methods.

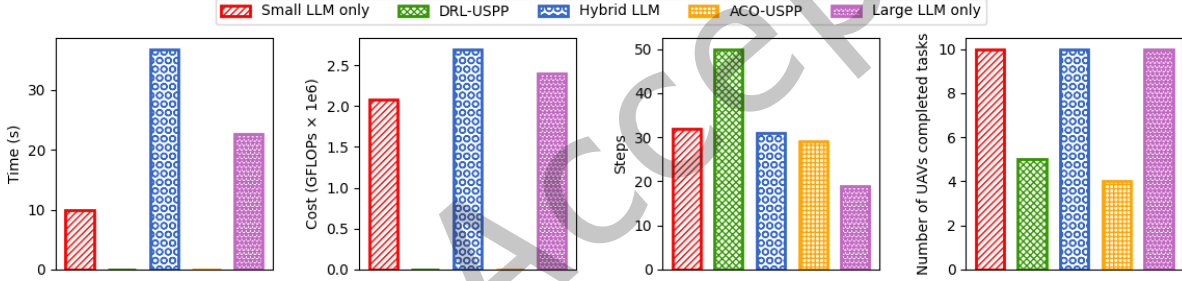


Fig. 11. The comparison of path planning performance of different methods.

results shown in Fig. 11, it indicate that traditional methods achieve lower system time and computational cost, but result in fewer UAVs completing tasks in complex path planning scenarios. Although ACO-USPP uses fewer decision steps than other methods, this is because UAVs that did not complete tasks were damaged early, causing premature termination of the swarm's path planning task. At the same time, we should acknowledge that although LLMs offer better interpretability and adaptability, they significantly increase computational consumption and processing time.

Second, to evaluate the learning ability and adaptability of different methods in dynamic scenarios, we randomly generated 20 EC scenarios and recorded the number of training iterations each method required to finish the task after UAV failures. Additionally, we recorded the number of UAVs completing the path planning tasks when the maximum training iterations were reached. The experimental results in Fig.12 clearly show that ACO-USPP performs the worst, requiring the maximum training iterations for each EC scenario, and almost no UAVs completing tasks after the training. This is because the ant colony algorithm needs extensive iterations and pheromone updates to adapt to the new task, with only 100 iterations and limited local observations, it struggles to find effective paths. In addition, compared with DRL-USPP, the LLM-based methods demonstrate more stable decision-making performance and can adapt to new scenarios with minimal training. This advantage stems from our multi-stage deep thinking training, as well as the scene replay and learning mechanism, which enable rapid recognition and correction even in the case of inference errors. Finally, we note that methods based on large

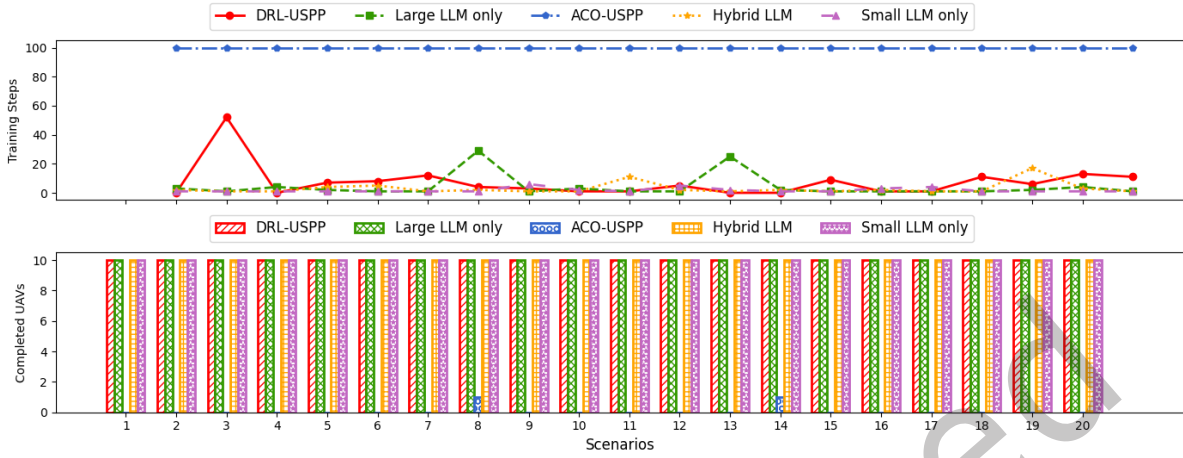


Fig. 12. The performance comparison of different methods in dynamic scenarios.

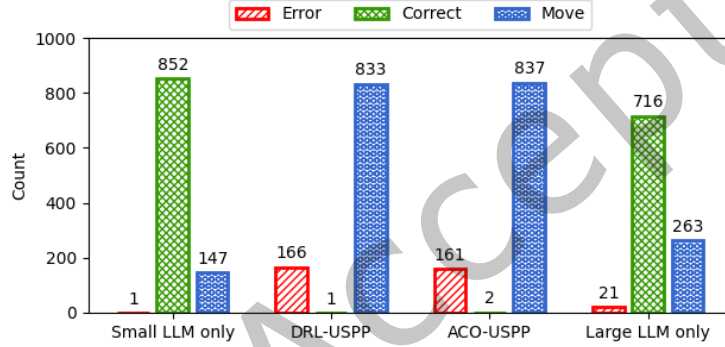


Fig. 13. The comparison of local movement decision-making performance across different methods.

LLMs may experience performance fluctuations. This is because they process a larger range of local information and require more reasoning steps, which increases the likelihood of being misled by certain tokens. Fortunately, the larger number of parameters of the large LLM also enhances the effectiveness of our proposed scenario replay and learning mechanism. As a result, after training, the large LLM-based methods demonstrate even more stable performance in subsequent scenarios compared to those using only small LLMs.

Finally, to evaluate the ability of different methods to generate movement decisions based on local information, we dynamically generated 1,000 samples and evaluated different methods. The evaluation based on these 1,000 samples is equivalent to the UAVs performing multiple movement decisions in a larger-scale environment, which can reflect their performance in uncertain scenarios. There is no need to evaluate the movement decision-making ability of the hybrid LLM scheme, as it is a combination of the large and small LLMs. Therefore, we only evaluate the large LLM and the small LLM separately. In this experiment, we will not train any methods further. Fig. 13 shows our evaluation results. It can be seen that traditional path planning algorithms not only have a higher probability of making incorrect movement decisions, but also rarely arrive at the optimal movement decision. Moreover, in dynamic environments, once DRL-USPP stops training, its performance becomes similar to ACO-USPP, indicating that traditional path planning algorithms struggle to perform well in highly dynamic scenarios. In contrast, LLM-based methods generate optimal local movement decisions in the vast majority of cases, demonstrating the strong adaptability of LLMs fine-tuned with multi-step deep thinking samples.

7 CONCLUSION

This paper investigates LLM-based swarm UAV path planning in edge computing scenarios under the context of Industry 5.0, aiming to address the limitations of existing methods in interpretability and flexibility, and each UAV generates movement decisions based solely on local observations. Our goal is to enable more UAVs to successfully complete path planning tasks while minimizing the time and computational cost of the edge system. To address the limitations of simple chain-of-thought prompting, we simulate the multi-step deep thinking and reasoning process of human beings to propose an algorithm for generating UAV path planning datasets for LLMs' fine-tuning. Specifically, we first design a UAV swarm path planning prompt template and a multi-step deep thinking template. Then, it combines a DRL-like reward modeling approach and greedy search to find the optimal move position. Finally, we dynamically embed UAV state information, the optimal movement position, and the reason why the UAV can't move to other positions into the prompt and multi-step deep thinking templates to construct fine-tuning samples. After multiple rounds of sample generation, we obtain a fine-tuning dataset for LLMs. Then, to address inference errors caused by misleading tokens during LLM inference, we design a scene memory and replay mechanism, enabling the system to continuously learn and improve over time. Experimental results show that the proposed LLM-based UAV path planning algorithm achieves the highest task completion rate among all compared methods, while maintaining a competitive task processing time. Besides, the LLMs in the system can learn and optimize inference capabilities even in the presence of inference errors. In future work, we will explore distributed collaborative path planning and operation mechanisms for multi-UAV swarms, and investigate methods to further reduce the computational cost of LLMs and enhance system resilience.

REFERENCES

- [1] 3GPP. 2021. *3GPP TR 38.901: Study on channel model for frequencies from 0.5 to 100 GHz*. Technical Report 38.901. 3GPP. Version 16.1.0.
- [2] Unsloth AI. 2024. Unsloth: Accelerating AI Experimentation. <https://github.com/unslothai/unsloth>. Accessed: 2024-05-26.
- [3] Artificial Analysis Team. 2024. Artificial analysis. <https://artificialanalysis.ai/>. Accessed: 2024-06-20.
- [4] Zekun Bai, Hui Pang, Zhaonian He, Bin Zhao, and Tong Wang. 2024. Path planning of autonomous mobile robot in comprehensive unknown environment using deep reinforcement learning. *IEEE Internet of Things Journal* (2024).
- [5] Cesar Briso, Cesar Calvo, Zhuangzhuang Cui, Lei Zhang, and Youyun Xu. 2020. Propagation measurements and modeling for low altitude UAVs from 1 to 24 GHz. *IEEE Transactions on Vehicular Technology* 69, 3 (2020), 3439–3443.
- [6] Jiao Chen, Suyan Dai, Fangfang Chen, Zuohong Lv, Jianhua Tang, and Laiquan Han. 2024. Edge-cloud collaborative motion planning for autonomous driving with large language models. In *2024 IEEE 24th International Conference on Communication Technology (ICCT)*. IEEE, 185–190.
- [7] Qingyu Chen, Yan Hu, Xueqing Peng, Qianqian Xie, Qiao Jin, Aidan Gilson, Maxwell B Singer, Xuguang Ai, Po-Ting Lai, Zhizheng Wang, et al. 2025. Benchmarking large language models for biomedical natural language processing applications and recommendations. *Nature communications* 16, 1 (2025), 3280.
- [8] Marwan Dhuheir, Aiman Erbad, Bechir Hamdaoui, Samir Belhaouari, Mohsen Guizani, and Thang X Vu. 2025. Multi-Agent Meta Reinforcement Learning for Reliable and Low-Latency Distributed Inference in Resource-Constrained UAV Swarms. *IEEE Access* (2025).
- [9] Junchao Fan, Xuyang Lei, Xiaolin Chang, Jelena Miši, Vojislav B Miši, and Yingying Yao. 2025. Less is More: A Stealthy and Efficient Adversarial Attack Method for DRL-based Autonomous Driving Policies. *IEEE Internet of Things Journal* (2025).
- [10] Xinzhen Feng, Jun Wu, Ali Kashif Bashir, Jianhua Li, Ao Shen, and Mohammad Dahman Alshehri. 2023. Vulnerability-aware task scheduling for edge intelligence empowered trajectory analysis in intelligent transportation systems. *IEEE Transactions on Intelligent Transportation Systems* 24, 4 (2023), 4661–4670.
- [11] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [12] Yixue Hao, Long Hu, and Min Chen. 2023. Joint Sensing Adaptation and Model Placement in 6G Fabric Computing. *IEEE Journal on Selected Areas in Communications* 41, 7 (2023), 2013–2024. doi:10.1109/JSAC.2023.3280968
- [13] Yixue Hao, Jiaxi Wang, Dongkun Huo, Nadra Guizani, Long Hu, and Min Chen. 2023. Digital Twin-Assisted URLLC-Enabled Task Offloading in Mobile Edge Network via Robust Combinatorial Optimization. *IEEE Journal on Selected Areas in Communications* 41, 10 (2023), 3022–3033. doi:10.1109/JSAC.2023.3310051
- [14] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. 2024. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*. 36–41.

- [15] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [16] Yaqi Hu, Fei Wang, Dongdong Ye, Maoqiang Wu, Jiawen Kang, and Rong Yu. 2025. LLM-Based Misbehavior Detection Architecture for Enhanced Traffic Safety in Connected Autonomous Vehicles. *IEEE Transactions on Vehicular Technology* (2025).
- [17] Xiaoge Huang, Xuan Yang, Qianbin Chen, and Jie Zhang. 2021. Task offloading optimization for UAV-assisted fog-enabled Internet of Things networks. *IEEE Internet of Things Journal* 9, 2 (2021), 1082–1094.
- [18] Fengcai Huo, Shuai Zhu, Hongli Dong, and Weijian Ren. 2024. A new approach to smooth path planning of Ackerman mobile robot based on improved ACO algorithm and B-spline curve. *Robotics and Autonomous Systems* 175 (2024), 104655.
- [19] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [20] Woojin Lee, Babar Shahzaad, Balsam Alkouz, and Athman Bouguettaya. 2024. Reactive composition of uav delivery services in urban environments. *IEEE Transactions on Intelligent Transportation Systems* (2024).
- [21] Junyi Li, Tianyi Tang, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2024. Pre-trained language models for text generation: A survey. *Comput. Surveys* 56, 9 (2024), 1–39.
- [22] Kai Li, Wei Ni, Xin Yuan, Alam Noor, and Abbas Jamalipour. 2023. Exploring graph neural networks for joint cruise control and task offloading in UAV-enabled mobile edge computing. In *2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring)*. IEEE, 1–6.
- [23] Yang Li, Yanqiang Wu, Xinyu Xue, Xuemei Liu, Yang Xu, and Xinghua Liu. 2024. Efficiency-first spraying mission arrangement optimization with multiple UAVs in heterogeneous farmland with varying pesticide requirements. *Information Processing in Agriculture* 11, 2 (2024), 237–248.
- [24] Zhiyang Liu, Liuhuan Li, Xiao Zhang, Wan Tang, Zhen Yang, and Ximin Yang. 2025. Considering both energy effectiveness and flight safety in UAV trajectory planning for intelligent logistics. *Vehicular Communications* 52 (2025), 100885.
- [25] Ruikun Luo, Zujia Zhang, Qiang He, Mengxi Xu, Feifei Chen, Xiaohai Dai, Song Wu, and Hai Jin. 2025. Cost-Effective Edge Data Caching With Failure Tolerance and Popularity Awareness. *IEEE Transactions on Mobile Computing* 24, 6 (2025), 5357–5369.
- [26] Yiming Miao, Kai Hwang, Di Wu, Yixue Hao, and Min Chen. 2022. Drone swarm path planning for mobile edge computing in industrial internet of things. *IEEE Transactions on Industrial Informatics* 19, 5 (2022), 6836–6848.
- [27] Zhenyu Na, Yue Liu, Jingcheng Shi, Chungang Liu, and Zihao Gao. 2020. UAV-supported clustered NOMA for 6G-enabled Internet of Things: Trajectory planning and resource allocation. *IEEE Internet of Things Journal* 8, 20 (2020), 15041–15048.
- [28] Sameer Abdullah Nooh. 2025. Optimizing Low Carbon Sustainable Environmental Monitoring With Consumer Technology: An IoT-Driven Federated Learning Approach for Edge Computing Optimization. *IEEE Transactions on Consumer Electronics* (2025).
- [29] Aryan Nikul Patel, Gautam Srivastava, Praveen Kumar Reddy Maddikunta, Ramalingam Murugan, Gokul Yenduri, and Thippa Reddy Gadekallu. 2024. A trustable federated learning framework for rapid fire smoke detection at the edge in smart home environments. *IEEE Internet of Things Journal* (2024).
- [30] Li Ping Qian, Hongsen Zhang, Qian Wang, Yuan Wu, and Bin Lin. 2022. Joint multi-domain resource allocation and trajectory optimization in UAV-assisted maritime IoT networks. *IEEE Internet of Things Journal* 10, 1 (2022), 539–552.
- [31] Hongchun Qu, Chaofang Zheng, Hao Ji, Kallol Barai, and Yong-Jiang Zhang. 2024. A fast and efficient approach to estimate wild blueberry yield using machine learning with drone photography: Flight altitude, sampling method and model effects. *Computers and Electronics in Agriculture* 216 (2024), 108543.
- [32] Sarah Sandmann, Sarah Riepenhausen, Lucas Plagwitz, and Julian Varghese. 2024. Systematic analysis of ChatGPT, Google search and Llama 2 for clinical decision support tasks. *Nature Communications* 15, 1 (2024), 2050.
- [33] Andrey V Savkin, Chao Huang, and Wei Ni. 2022. Joint multi-UAV path planning and LoS communication for mobile-edge computing in IoT networks with RISs. *IEEE Internet of Things Journal* 10, 3 (2022), 2720–2727.
- [34] Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2020. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 8815–8821.
- [35] Xue Sun and Yu Song. 2025. Unlocking the Synergy: Increasing productivity through Human-AI collaboration in the industry 5.0 Era. *Computers & Industrial Engineering* 200 (2025), 110657.
- [36] Julio Terven, Daniel M Cordova-Esparza, Jos   A Romero-Gonz  lez, et al. 2025. A comprehensive survey of loss functions and metrics in deep learning. *Artificial Intelligence Review* 58 (2025), 195.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [38] George A Vouros. 2022. Explainable deep reinforcement learning: state of the art and challenges. *Comput. Surveys* 55, 5 (2022), 1–39.
- [39] Jinhai Wang, Changqing Du, Fuwu Yan, Xuexue Duan, Min Hua, Hongming Xu, and Quan Zhou. 2024. Energy Management of a Plug-in Hybrid Electric Vehicle Using Bayesian Optimization and Soft Actor-Critic Algorithm. *IEEE Transactions on Transportation Electrification* (2024).

- [40] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2024. Parameter-efficient fine-tuning in large models: A survey of methodologies. *arXiv preprint arXiv:2410.19878* (2024).
- [41] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2025. Parameter-efficient fine-tuning in large language models: a survey of methodologies. *Artificial Intelligence Review* 58, 8 (2025), 227.
- [42] Nuanlai Wang, Shanchen Pang, Xiaofeng Ji, Min Wang, Sibao Qiao, and Shihang Yu. 2024. Intelligent Driving Task Scheduling Service in Vehicle-Edge Collaborative Networks Based on Deep Reinforcement Learning. *IEEE Transactions on Network and Service Management* (2024).
- [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [44] Xianglin Wei, Lingfeng Cai, Nan Wei, Peng Zou, Jin Zhang, and Suresh Subramaniam. 2023. Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing. *IEEE Internet of Things Journal* 10, 14 (2023), 12826–12838.
- [45] Hongyue Wu, Mengchen Wu, Wen Peng, Shizhan Chen, and Zhiyong Feng. 2023. Its: Improved tabu search algorithm for path planning in uav-assisted edge computing systems. In *2023 IEEE International Conference on Web Services (ICWS)*. IEEE, 340–349.
- [46] Wenjing Xiao, Xin Ling, Miaojiang Chen, Junbin Liang, Salman A Alqahtani, and Min Chen. 2024. MVPOA: A Learning-Based Vehicle Proposal Offloading for Cloud-Edge-Vehicle Networks. *IEEE Internet of Things Journal* (2024).
- [47] Wenjing Xiao, Chenglong Shi, Miaojiang Chen, Zhiqian Liu, Min Chen, and H. Herbert Song. 2025. GraphEdge: Dynamic graph partition and task scheduling for GNNs computing in edge network. *Information Fusion* (2025), 103329.
- [48] Qingzhong Yan, Xiang Wu, Jiacun Wang, Giancarlo Fortino, Francesco Pupo, and Mingfeng Yin. 2025. EGCARL: A PPO-based reinforcement learning method with expert guidance and dynamic rewards for autonomous driving. *Information Fusion* (2025), 103606.
- [49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [50] Liang Yao, Fan Liu, Shengxiang Xu, Chuanyi Zhang, Xing Ma, Jianyu Jiang, Zequan Wang, Shimin Di, and Jun Zhou. 2024. UEMM-Air: Make Unmanned Aerial Vehicles Perform More Multi-modal Tasks. *arXiv e-prints* (2024), arXiv–2406.
- [51] Yao Yao, Zuchao Li, and Hai Zhao. 2024. Gkt: A novel guidance-based knowledge transfer framework for efficient cloud-edge collaboration llm deployment. *arXiv preprint arXiv:2405.19635* (2024).
- [52] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMCC2-NIPS)*. IEEE, 36–39.
- [53] Chao Zeng, Songwei Liu, Yusheng Xie, Hong Liu, Xiaojian Wang, Miao Wei, Shu Yang, Fangmin Chen, and Xing Mei. 2025. Abq-llm: Arbitrary-bit quantized inference acceleration for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 22299–22307.
- [54] Ye Zeng, Li Qiao, Zhen Gao, Tong Qin, Zhonghuai Wu, Emad Khalaf, Sheng Chen, and Mohsen Guizani. 2024. CSI-GPT: Integrating generative pre-trained transformer with federated-tuning to acquire downlink massive MIMO channels. *IEEE Transactions on Vehicular Technology* (2024).
- [55] Guobin Zhang, Xuqiang Chen, Guojun Han, Chang Liu, and Jiawen Kang. 2025. Blockchain-Based Distributed Resource Management for Delay Minimization in UAV-Assisted Vehicular Networks. *IEEE Transactions on Vehicular Technology* (2025).
- [56] Wei Zhang, Lu Hou, Yichun Yin, Lifeng Shang, Xiao Chen, Xin Jiang, and Qun Liu. 2020. Ternarybert: Distillation-aware ultra-low bit bert. *arXiv preprint arXiv:2009.12812* (2020).
- [57] Ye Zhang, Wang Zhao, Jingyu Wang, and Yuan Yuan. 2024. Recent progress, challenges and future prospects of applied deep reinforcement learning: A practical perspective in path planning. *Neurocomputing* 608 (2024), 128423.
- [58] Wentao Zhao, Wenpeng Jing, Zhaoming Lu, and Xiangming Wen. 2024. Edge and terminal cooperation enabled llm deployment optimization in wireless network. In *2024 IEEE/CIC International Conference on Communications in China (ICCC Workshops)*. IEEE, 220–225.
- [59] Kechen Zheng, Rongwei Luo, Xiaoying Liu, Jiefan Qiu, and Jia Liu. 2024. Distributed DDPG-based resource allocation for age of information minimization in mobile wireless-powered Internet of Things. *IEEE Internet of Things Journal* (2024).
- [60] Hao Zhou, Chengming Hu, Dun Yuan, Ye Yuan, Di Wu, Xue Liu, Zhu Han, and Jianzhong Zhang. 2024. Generative ai as a service in 6g edge-cloud: Generation task offloading by in-context learning. *IEEE Wireless Communications Letters* (2024).
- [61] Xun Zhou, Guangjie Han, Guoxiong Zhou, Yongfei Xue, Mingjie Lv, and Aibin Chen. 2025. Hybrid DQN-Based Low-Computational Reinforcement Learning Object Detection With Adaptive Dynamic Reward Function and ROI Align-Based Bounding Box Regression. *IEEE Transactions on Image Processing* (2025).

Received 23 June 2025; revised 6 September 2025; accepted 22 October 2025